WaveVis inverse-sheet architecture for rounded breaking-wave linkage arrays

Alan N. Pham

AO Labs, WaveVis simulator, inverse deformation sheet record

WaveVis is an inverse computational-design record for a rectangular linkage sheet that is being driven toward one localized plunging-wave deformation while keeping the mechanism visible. This PDF is not allowed to be a decorative paper, a candidate gallery, or a misleading overlay. It is the reconstruction manual for the project: the target references, coordinate frames, source files, equations, topology constraints, render/display split, validation gates, failure ledger, deployment boundary, and next iteration protocol needed for a fresh engineer to continue from source truth. The accepted reference family is the latest supplied rounded breaking-wave side line, the June 24 smooth gridded overview/sequence, the June 28 stage/cross-section sheet, and the two retained ocean-curl photographs used only as secondary water-form references. The rejected family is equally explicit: swept tunnels, side walls, skinny center-strip sails, triangular cuts, pointy side profiles, bowls, dimples, dark pockets, under-overhang support necks, profile-only traces, hidden mechanisms, fake filler bridges, old candidate screenshots, wrong or irrelevant stills, and any figure that makes the wrong geometry look authoritative. Current publication state is Candidate876: the readable side reference is the segmented rounded supplied line; the terminal sampler slows the lip return so the lower curl remains inside the rounded hook before releasing to the flat tail; the terminal fold/lift envelopes were broadened to reduce the support-cone tendency without passing the broad-wall blockers; `view=front` samples a crest-section window from the same three-dimensional reference field; `scripts/check-inverse-sheet.cjs` passes the rounded-cap, flat-perimeter, no-dimple, no-wall, terminal-taper, tunnel-localization, and full visible X-mechanism gates; the rebuilt PDF was copied into the public proof paths; and the July 1, 2026 live route check served bundle `index-SPBHkj_l.js` on both the fallback and standalone HTTP routes. This is still not a reference-match completion claim. The side profile is no longer the rejected pointy profile, but the 3D support-cone read remains an open visual-match boundary until a future rendered candidate removes it.

Fresh-chat use. Read this PDF before changing WaveVis. Then inspect the current source, the live route, the latest handoff, and the rendered artifact. If any of those disagree, the live/source/rendered state wins over this prose until the PDF is rebuilt and reverified. A future chat should be able to reconstruct the model and the iteration state from this record without relying on old chat memory.

Contents

1 Introduction

WaveVis is an inverse-design simulator for a deployable mechanism. It takes a target surface intent, samples it on a rectangular sheet, and renders the resulting cell/connector mechanism rather than only a smooth mesh. That places it between several technical families: deformable-body, cloth, shell, rod, and mass-spring simulation where surfaces and slender members are discretized, regularized, and advanced under mechanical constraints (*1–5*); spacetime, constraint-projection, projective-dynamics, and differentiable-simulation methods where feasibility is enforced by explicit objective terms, local/global corrections, or gradients (*6–10*); computational mechanism design where actuator placement, linkage topology, compliant flexures, and contact constraints are part of the design object (*11–16*); and inverse/topology design of mechanisms, inflatables, auxetics, and metamaterials where a desired deformation field must be made compatible with a manufacturable structure (*17–23*). WaveVis is not a solved member of any of those literatures. It is an active reconstruction record for one specific mechanism question: can a rectangular linkage sheet be made to read as a localized plunging wave while keeping its full kinematic graph visible?

The design problem is stricter than drawing a curve or matching a screenshot. A side outline can look acceptable while the actual three-dimensional sheet has a wall across the underside, a cavity where the lip should be open, an extruded barrel, or zig-zagging linkage legs that no longer read as a mechanism. Conversely, a mechanism can satisfy a local numerical check while failing the human outcome: a full rectangular sheet with a localized plunging wave. WaveVis must therefore treat the visible artifact, target field, kinematic topology, URL state, source checks, paper figures, and public deployment as one coupled system.

The central correction captured here is that a breaking wave is not made by sweeping one two-dimensional profile sideways. A swept profile creates a tunnel, canopy, or half-pipe. The required object is a localized deformation field on a flat sheet:

$$\Phi : (x, y) \mapsto \big(x + \Delta_x(x, y),\ y + \Delta_y(x, y),\ \Delta_z(x, y)\big),$$

where the displacement is concentrated near the middle span, gradually fades toward the lateral sides, and is exactly zero on the perimeter. The curl must be strongest near the centerline and weaker toward the sides. This spatial falloff is part of the target shape, not a rendering trick.

The current hard failure is not subtle. A rendered state showed an inward support neck and dimple below the curling lip. That shape class is banned because it means the simulator has built a local support column or bowl where the reference requires an open throat. No overlay, centerline plot, paper caption, or metric-only pass may convert that failure into evidence.

1.1 Reference geometry

The design reference is the latest supplied rounded side-profile line plus the gridded breaking-wave reference sheets and retained ocean-curl photographs: a smooth, localized volume that rises out of a larger flat sheet, grows into a broad rounded crest, curls into a smaller inward spiral, and finishes with a smooth downturned tapered lip while leaving an open underside and a long low return. These reference frames are not decorative. They are the visual contract for the simulator's target geometry. A successful WaveVis state should read as a simplified linkage approximation of this water form: the outer radius is large and smooth, the inner curl radius is smaller, the lip projects forward before curling down, the front/cross-section view is rounded rather than pointed, and the side field fades away instead of forming a constant barrel. This is deliberately stricter than matching a side-profile curve: the full linkage array in side and isometric views must preserve the open curl without side walls, erratic zig-zags, disconnected cells, or hidden filler geometry.

Current verification boundary. This document records the target, architecture, and required gates. It does not by itself prove that the current generator has matched the supplied references. The rendered full-array artifact remains the source of truth. If the page, screenshots, or live route do not visibly match the localized plunging-wave reference family while preserving linkage cells and flat perimeter, the correct state is "open visual mismatch," not "reference matched."

Figure 1. Primary supplied reference geometry. The target form is a localized plunging water sheet with a rounded crest, open underside, and forward/downward lip. The WaveVis sheet must approximate this geometry while preserving flat perimeter boundary nodes and visible linkage cells.

Figure 2. Secondary supplied open-throat reference. The image is used only to define the underside condition: the lip is suspended, the throat remains open, and no side wall, support neck, or dark pocket may be used to close the curl.

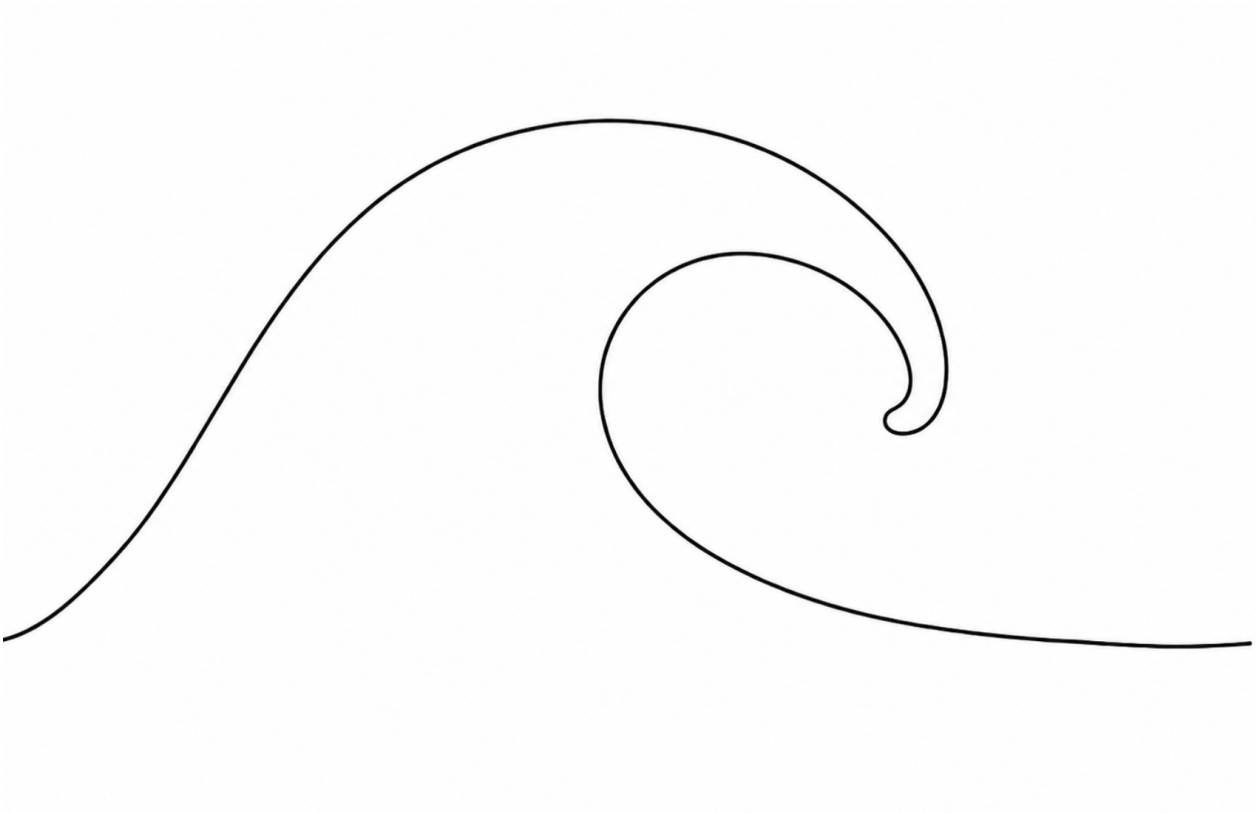

Figure 3. Latest supplied side-profile reference. This image sharpens the visible target: broad rounded outer arch, smaller inward curl, smooth downturned tapered tip, smooth curved inner throat, and long low return. A pointy side profile is explicitly wrong. This is a shape reference only; the three-dimensional WaveVis render must still preserve the full linkage sheet rather than becoming a two-dimensional trace or silhouette.

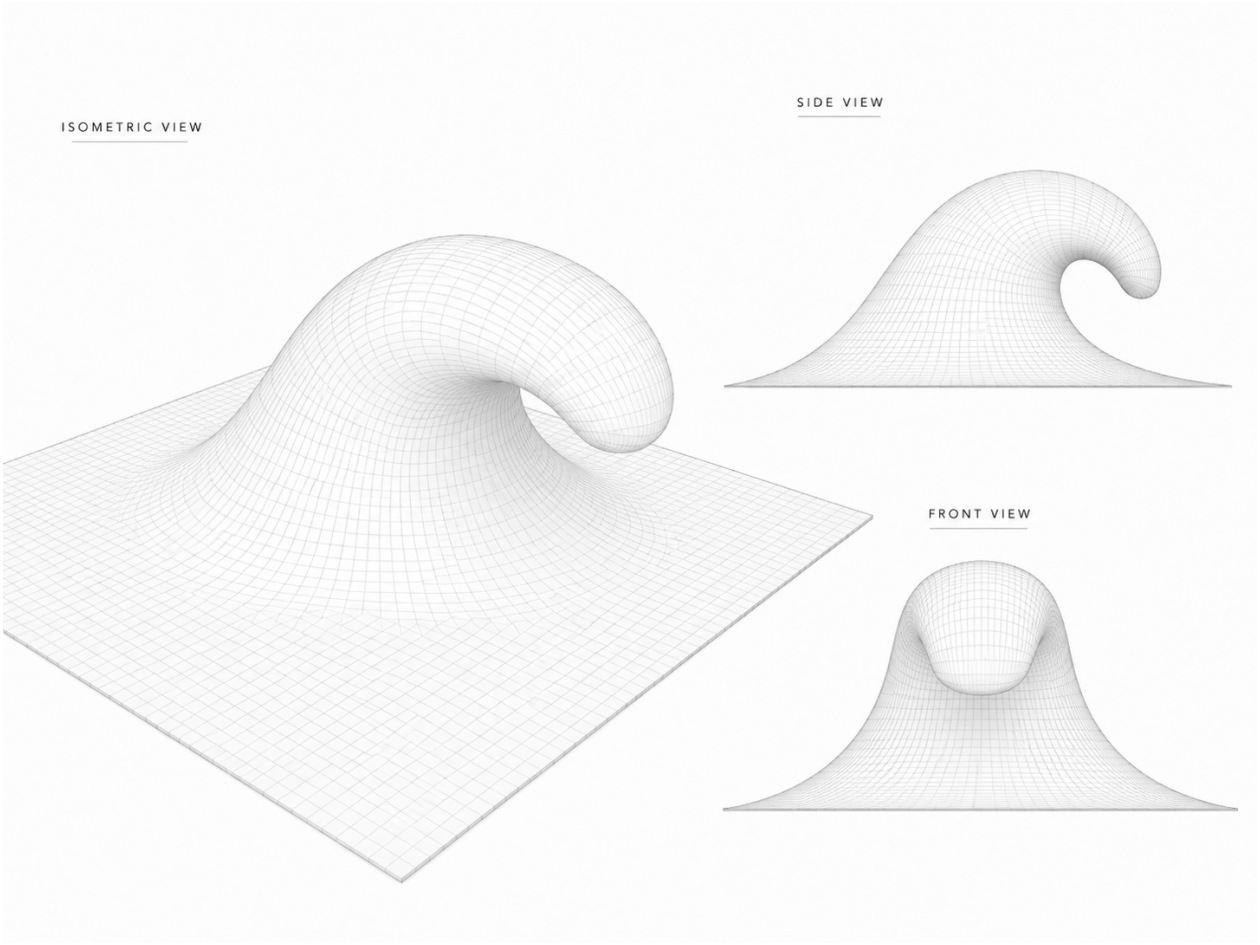

Figure 4. June 24 supplied smooth gridded reference overview. Alan re-supplied this image and said "remember." It is now a preserved project reference for the desired visual grammar: a continuous square sheet surface, rounded rising face, tube-like crest, downturned lip, open clean throat, and centered front view. A jagged linkage trace, side-wall tunnel, or metric-only improvement does not satisfy this reference.

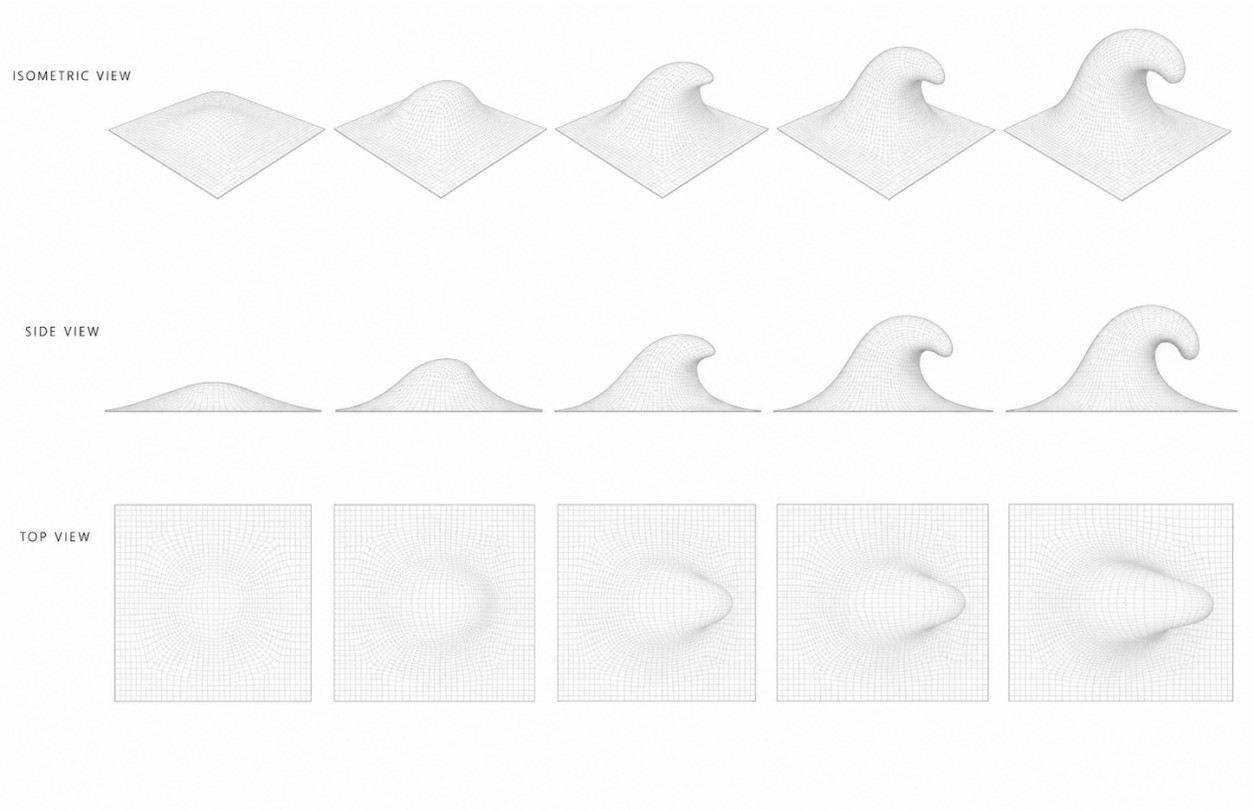

Figure 5. June 24 supplied smooth gridded reference sequence. The sequence records the intended progression across isometric, side, and top views: the deformation grows from a flat square sheet into a localized rounded wave with a curled lip while the top view remains a rounded localized body, not a triangular fan or swept strip.

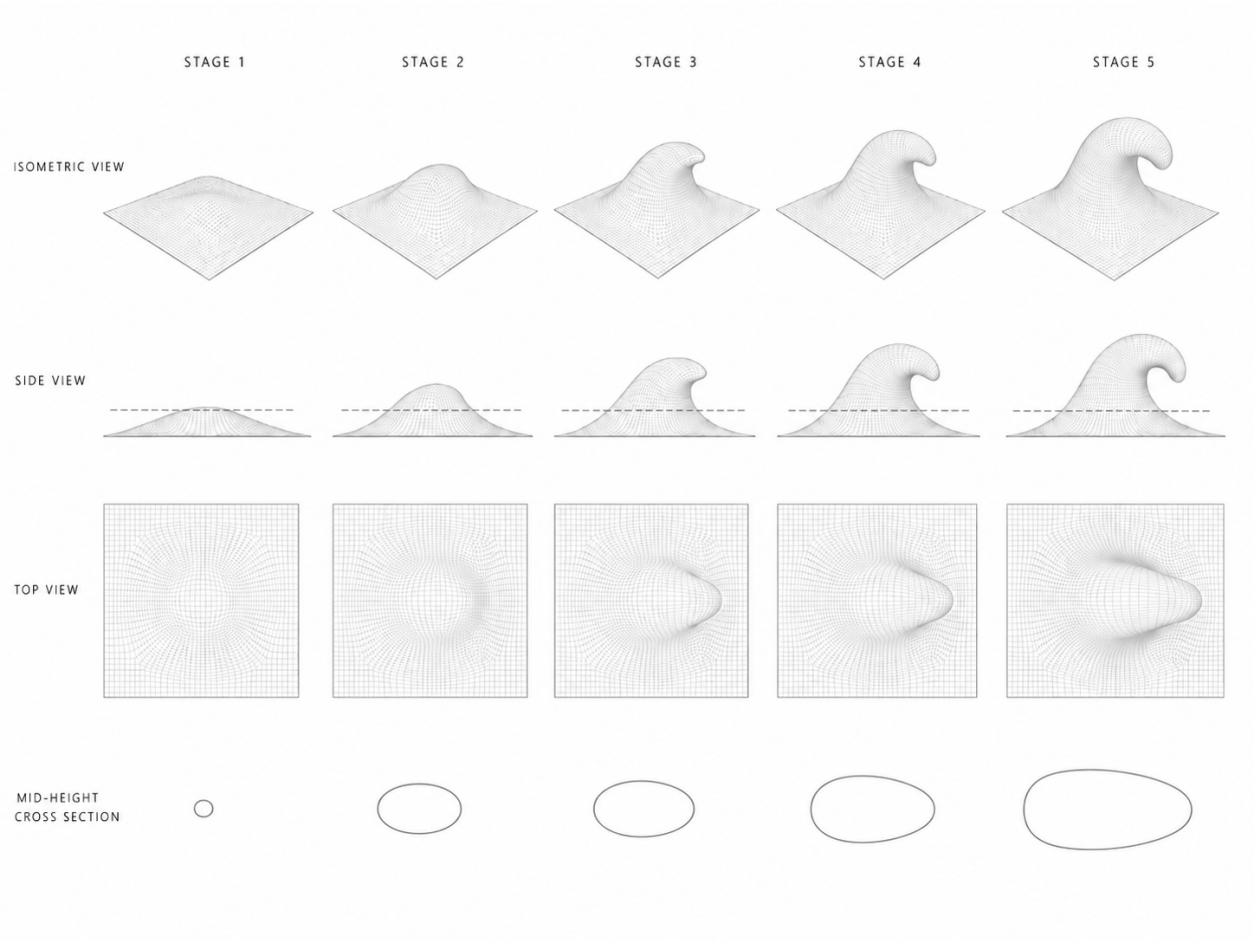

Figure 6. June 28 supplied stage sequence with an explicit mid-height cross-section row. The bottom row tightens the plan-footprint target: the active body should grow into one smooth oval or teardrop-like section through the curl, not a pointed spear, pinched hourglass, terminal stripe, or pasted helper contour. This reference is now part of the WaveVis acceptance set.

2 Start here for a new WaveVis chat

If this PDF is the only WaveVis context available, start with this section and then inspect the live artifact. The current project state is:

- **Verified public route:** `https://aolabs.io/wavevis/`. This route serves the fallback bundle that Alan actually uses.
- **Working standalone route:** `http://wavevis.aolabs.io/`. This route serves the current WaveVis bundle. The HTTPS version remains a separate certificate boundary.
- **Preferred HTTPS route still blocked:** `https://wavevis.aolabs.io/`. At the time of this record, HTTPS validation can fail because the custom-domain certificate is not valid for that

hostname. Do not treat the HTTPS subdomain as the verified route until a fresh certificate check passes.

- **Visible PDF action:** the WaveVis header links to the paper file under `/proofs/`. File: `wavevis-system-architecture.pdf`. The old connector proof remains a source artifact, not the primary user-facing paper action.
- **Current source root:** workspace path `_apps/wavevis.aolabs.io/`.
- **Public fallback mirror:** workspace path `_apps/aolabs-site/wavevis/`. A WaveVis deploy to its own `gh-pages` branch does not automatically update the `aolabs.io/wavevis/` fallback unless the built files are mirrored there and the hub site is pushed.
- **Current architectural direction:** curated supplied-reference field localized into the inverse-sheet model, plus a separate readable display surface for visual inspection. Manual xz and xy profile editors are not the active path.
- **Current branch state:** Candidate876 is the active local source state. It changes `src/LatticeViewer3D.tsx` and `scripts/check-inverse-sheet.cjs`: segmented rounded side trace, rounded lip-tip control points, slower terminal profile sampler, broadened terminal fold/lift envelopes, sectioned front view, and updated no-point/no-wall/no-terminal-foot gates. Candidate841–Candidate875 are rejected or superseded diagnostics that explain the failures being blocked: support-neck dimples, broad roof/curtain, triangular cut/fin, side-wall tunnel, pointy side profile, full-length front projection, and squared terminal foot.
- **Current verification state:** the local inverse-sheet checker `scripts/check-inverse-sheet.cjs` passes; `npm run build` passes; browser screenshots exist in:

```
_verification/
browser-20260701-
candidate876-broader-terminal/
```

The side view now follows the rounded supplied profile instead of the rejected pointy profile, and the front view remains a rounded crest-section rather than a spike. The rebuilt PDF, fallback route, and standalone HTTP route were checked on July 1, 2026 for bundle `index-SPBHkj_l.js` and the matching public PDF copy. The preferred HTTPS subdomain remains certificate-blocked. The isometric support-cone read is still open visual debt.

- **Correct acceptance target:** the supplied June 24 smooth breaking-wave reference family and June 28 stage/cross-section reference. The wrong target is any internal trace diagram, centerline overlay, profile-only silhouette, or helper contour that cannot reconstruct the mechanism.
- **Current mechanism contract:** every interior cell is one black square body with four straight legs; opposite leg pairs are equal length and collinear through the cell center; every spherical connector node is a physical two-cell joint with exactly two attached leg uses; crossing diagonal families are separate joints rather than four-cell connector collisions.
- **Do-not-claim boundary:** do not claim reference match, paper current, or public completion from a geometry log, a compiled PDF, a local screenshot, an internal overlay, or a smoother profile string. The rendered full mechanism is the acceptance object.

2.1 What the next agent must not repeat

The repeated failures that this document is meant to prevent are:

1. Do not make a swept tunnel, ribbon, canopy, roof, half-pipe, bridge, or extruded strip.
2. Do not form a bowl, dimple, cavity, pucker, under-overhang throat bite, or side wall that covers or pinches the underside of the curl.
3. Do not replace Side with a profile-only debug view. Side is a side camera view of the full three-dimensional sheet.
4. Do not hide, ghost, clip, or remove linkage cells because they look bad. Fix the geometry.
5. Do not remove the full array, one-center X cells, exact collinear equal opposite pairs, two-use physical connector joints, or two-cell linkage truth while chasing a prettier outline.
6. Do not wire the lip directly to the ground or draw filler geometry across the open underside.
7. Do not make `position` or `steer` shape controls. They are rigid post-shape transforms.
8. Do not call the work done from code intent, automated checks, a nicer profile line, or a local-only screenshot.
9. Do not let the top view collapse into a big triangle, wedge, or V fan while the side view looks acceptable.

10. Do not paste a mid-height contour, helper ellipse, bridge, or cross-section overlay into the product view to fake the June 28 cross-section row. Fix the surface projection and preserve the X-only proof path.

2.2 Current state ledger

This subsection replaces the old Candidate785 figure gallery. Those screenshots are historical diagnostics, not final paper evidence. The paper must not show a stale candidate as if it were the correct geometry. The current ledger is:

Class	Current source	Paper/display rule
Accepted references	Latest supplied rounded side drawing, June 24 smooth gridded overview/sequence, June 28 stage/cross-section reference, and retained ocean-curl photos used only for water-form context.	Keep as target figures and cite them as references, not as achieved WaveVis output.
Rejected display asset	Profile-trace overlay image and non-target stills.	Removed from tracked proof and public figure assets; do not use internal trace overlays or visually irrelevant stills as paper figures.
Rejected candidate states	Candidate785 and Candidate841–Candidate875 screenshots.	Historical diagnostics only. Do not describe them as accepted renders after Alan identified support-neck, pointy-profile, front-fin, wall, blob, cone, and terminal-foot families.
Active local candidate	Candidate876 viewer and checker changes: segmented rounded side trace, rounded lip-tip anchors, terminal sampler slowdown 0.064, broadened terminal fold/lift envelopes, sectioned front view from the shared 3D reference field, no-point/no-wall/no-terminal-foot gates, flat perimeter, and visible mechanism locks.	Geometry/build/browser/PDF verification and live fallback/standalone HTTP route checks exist. The side profile is improved; exact 3D reference match is still open because the isometric support-cone read remains visible.
Mechanism invariant	One-center four-leg X cells with exact opposite-pair collinearity/equal length and physical two-use connector nodes.	Keep as a hard gate and cite source/checker values after each accepted candidate.
Open boundary	Exact reference-family match and public/fallback route parity.	Blocks final paper claims, deployment claims, and Progress closure until live routes and PDF renders are verified.

Table 1. WaveVis state ledger as of July 1, 2026. The paper tracks which information is reference truth, rejected history, active source state, and still-open defect state.

The only current figures that belong in the paper by default are target references and mechanism-proof figures that still define the actual constraint. Candidate screenshots belong only when they are labeled as rejected or diagnostic and they help the next iteration avoid a known failure.

2.3 Figure and asset hygiene

WaveVis now treats paper figures as source evidence, not decoration. A file can appear in this PDF only if it belongs to one of four classes:

1. a target reference that defines the accepted geometry family;
2. a current accepted render that passed source checks, browser inspection, and route verification;
3. a mechanism proof figure that defines a hard constraint still used by the source;
4. a clearly labeled rejected diagnostic that explains a failure mode future iterations must avoid.

Profile-trace overlays, centerline-only comparisons, stale candidate screenshots, unrelated stills, helper contours, and any image whose purpose is to make the current source look more correct than the rendered mechanism must stay out of the paper and the public proof folder. When an asset is removed for this reason, the removal is part of the source-of-truth record rather than cleanup churn.

2.4 File map

221

File or route	Ownership
`src/latticeGeometry.ts`	Source of truth for the inverse sheet model, defaults, sanitization, target displacement field, morph behavior, metrics, and geometry sanity checks.
`src/LatticeViewer3D.tsx`	Source of truth for Canvas rendering, side/isometric/top full-sheet camera behavior, surface visibility, camera presets, visual layers, and the rendered collinear equal-pair X-cell and shared-joint layers.
`src/xCellMechanism.ts`	Source of truth for the connected X-cell display mechanism: one black-square cell body per lattice cell, four straight center-to-connector legs per cell, exact collinear/equal opposite pairs through the cell center, and spherical connector nodes with exactly two attached leg uses.
`src/TargetShapeControls.tsx`	Source of truth for the visible inverse-sheet controls: rows, columns, views, scale, morph, position, steer, display toggles, reset, export, and import.
`src/InverseSheetTab.tsx`	Source of truth for URL-state loading, tab order, view requests, import/export plumbing, and the Inverse Sheet first-tab behavior.
`scripts/check-inverse-sheet.cjs`	Main inverse-sheet regression checker. It compiles the TypeScript geometry and checks parameter semantics, lip curl, flatContribution, low-lip stability, lateral smoothness, no bowl pockets, terminal taper, startup URL coverage, and the collinear equal-pair X-cell render contract.
`scripts/check-geometry-invariants.cjs`	Mechanism-level invariant checker for rigid-cell/linkage behavior.
`proofs/wavevis-system-architecture.tex`	Source for this PDF. Update it when the public simulator contract, current state, or known failure modes change.
`proofs/figures/`	Project-defining reference images and mechanism proof figures used by this paper. Old current-state screenshots and trace overlays must stay out of the paper unless explicitly labeled as rejected diagnostics.
`dist/`	Built WaveVis static app after `npm run build`. Do not edit by hand.
`aolabs-site/wavevis/`	Public fallback mirror for `https://aolabs.io/wavevis/`. Copy the built `dist` contents here when the public fallback must update.

Table 2. Current source map. A new WaveVis chat should use these files rather than reconstructing behavior from old prompts.

2.5 Minimal continuation workflow

A fresh agent continuing WaveVis should do the following in order:

1. Read this PDF and `src/latticeGeometry.ts`, then inspect `src/LatticeViewer3D.tsx` and `src/TargetShapeControls.tsx`.
2. Open the verified public route `https://aolabs.io/wavevis/` and the current local route if a dev server is running.
3. Reproduce Alan's active preset from the URL before changing source. Do not judge a different default state.
4. Run `npm run check:geometry`. If it fails, fix source before visual tuning.
5. Render and inspect `Side`, `3D`, and `Top`. Side and 3D must both be full-sheet views with the curl visible.
6. Make the smallest source change that improves the human-facing breaking-wave outcome without regressing the mechanism invariants.
7. Re-run `npm run check:geometry` and `npm run build`.
8. Verify live/public output after deploy or fallback mirror update. A local screenshot is not the public artifact.
9. Update this PDF if the architecture, current state, source map, controls, or known failure classes changed.
10. Post a Progress event with complaint, issue, Codex change, observed source change, provenance, and commit ids.

3 Current implementation state

3.1 Defaults and target

The current inverse-sheet default is intentionally taller and sharper than earlier flat presets:

$$\begin{aligned}
\texttt{rows} &= 44, & \texttt{columns} &= 112, & \texttt{morph} &= 1,\\
\texttt{height} &= 15.25, & \texttt{horizontalOffset} &= 22.5, & \texttt{overhangWidth} &= 32,\\
\texttt{overhangAngleDeg} &= 120, & \texttt{overhangPosition} &= -0.15, & \texttt{profileScale} &= 0.60,\\
\texttt{smoothing} &= 1, & \texttt{lipSharpness} &= 0.50, & \texttt{wallSmoothness} &= 0.28,\\
\texttt{flatContribution} &= 0.35.
\end{aligned}$$

The app displays only the narrow active controls by default. Legacy inputs such as `height`, `overhang`, `lipDip`, `width`, `lipSharpness`, `groundTransition`, `wallSmoothness`, and `flatContribution` can still be parsed from URLs and JSON imports because older test links and screenshots use them. The visible startup path favors `scale`, `morph`, `position`, and `steer` to keep the main wave profile stable.

The default reference-wave sheet remains `columns` = 112. The sanitizer now allows `columns` ≥ 12 so X-cell proof routes can be inspected at low density, but compact column counts are not evidence of an exact reference-wave match. Dense-wave regression checks request the default column count explicitly; low-column screenshots are mechanism-readability proofs.

3.2 Target-field construction

The target is constructed in a canonical wave frame. The canonical frame removes `position` and `steer`; the shape is computed there; then steer and position are applied as rigid transforms afterward. This prevents the old bug where moving the wave also changed its profile.

The source sequence in `buildNodes` is:

1. build rest grid p^0_{ij};
2. compute core target positions from `targetFromDeformationField`;
3. pin perimeter nodes to rest;
4. apply span continuity smoothing for generated mode;
5. apply flat-contribution diffusion as a support apron, not as a flattening blend;

6. interpolate from rest to target with `morphGrowthAmount`;

7. build metrics, edges, quads, and dihedral pairs from the resulting grid.

The target function samples u along the wave field and v across span:

$$u = \frac{x - x_{\min}}{L}, \qquad v = \frac{y + S/2}{S}.$$

The displacement is zero outside the support field and on the perimeter. The support field expands with `smoothing` and `flatContribution`, but the core wave must remain the same shape.

3.3 Reconstruction-grade mathematical model

Let the rest sheet be a rectangular lattice with R rows and C columns. In source, $R = 44$, $C = 112$, rest sheet length $L_s = 80$, rest sheet span $S_s = 80$, wave-field length $L = 43$, and wave-field span $S = 43$. A node is

$$p_{ij}^0 = \begin{bmatrix} -L_s/2 + jL_s/(C-1) \\ -S_s/2 + iS_s/(R-1) \\ 0 \end{bmatrix}, \qquad 0 \le i < R, \quad 0 \le j < C.$$

Boundary nodes are hard pinned:

$$p_{ij}^* = p_{ij}^0 \quad \text{if} \quad i \in \{0, R-1\} \text{ or } j \in \{0, C-1\}.$$

For interior nodes, WaveVis maps the world point into a canonical wave frame by removing user placement:

$$\begin{aligned} \tilde{p} &= R_z(-\psi)\left(p^0 - a - \delta_x e_x\right) + a, \\ \psi &= \frac{\pi}{4}\,\mathrm{clamp}(\texttt{steer}, -1, 1), \\ \delta_x &= 0.045L\,\mathrm{clamp}(\texttt{position}, -1, 1). \end{aligned}$$

The canonical target is

$$\tilde{p}^* = \tilde{p} + D(\tilde{x}, \tilde{y}; \theta),$$

and the world target is restored by applying the inverse rigid placement. This separation is required because inverse-design controls should not silently change the target surface. It follows the same general separation used in graphics simulators between state variables, constraints, and display transforms (*7*,

9).

The active source contains two target/display paths that must not be confused:

1. **Model target path,** owned by `src/latticeGeometry.ts`. It generates node targets, metrics, and the mechanism-check state.
2. **Readable display path,** owned by `src/LatticeViewer3D.tsx`. It draws a reference-like gridded surface and mechanism layer for visual inspection. It is not allowed to replace the model target or hide invalid mechanism state.

Custom profile path

When `profileMode='custom'`, the model source uses serialized control points. The current default centerline source is:

$$(0,0),(0.04,0.008),(0.095,0.033),(0.17,0.105),(0.275,0.305),$$
$$(0.385,0.58),(0.5,0.82),(0.61,0.965),(0.69,1),(0.78,0.965),$$
$$(0.858,0.86),(0.922,0.704),(0.967,0.528),(0.985,0.384),$$
$$(0.965,0.3),(0.937,0.228),(0.919,0.16),(0.934,0.092),$$
$$(0.971,0.037),(1,0).$$

Those are normalized (x,z) anchors, not final mechanism coordinates. They are sampled, scaled, localized across the span, morph-interpolated, and then checked against the mechanism constraints. If a future candidate changes this profile, it must record the new point list, the reason for every moved anchor, and the visual failure it fixes.

Generated lip path

The generated path uses a ramp/crest/lip decomposition:

$$P(u) = \begin{cases} P_{\text{ramp}}(u), & 0 \le u < u_c, \\ P_{\text{crest}}(u), & u_c \le u < u_l, \\ P_{\text{lip}}(u), & u_l \le u \le 1. \end{cases}$$

In source, `moanaReferenceLipPoints` builds a normalized list of lip anchors controlled by `dip` and `sharp`. The anchors are smoothed into cubic Bezier segments, then sampled by normalized arc length:

$$q(\alpha) = B_k(t) \quad \text{where} \quad \sum_{\ell<k} \int_0^1 \left\| B'_\ell(s) \right\| \, \mathrm{d}s + \int_0^t \left\| B'_k(s) \right\| \, \mathrm{d}s = \alpha \, \ell_{\text{total}}.$$

The source samples this curve numerically using 18 subdivisions per cubic segment. A future exact reconstruction should replace that sampling with an analytic or higher-resolution arc-length inversion only if the rendered mechanism improves and the validation gates still pass.

Span localization

The readable display path uses a lateral envelope

$$E(s) = \cos^{1.42}\left(\frac{\pi}{2}|s|\right), \qquad s \in [-1, 1],$$

then uses powers of E near the terminal lip to localize the curl. The source intentionally makes the lip narrower than the broad ramp; otherwise a centerline curl becomes a swept tunnel. Candidate876 keeps the terminal localization bounded, but lowers the powers from the old center-spike setting so the lip does not collapse into a support cone:

$$\tau(t) = \mathrm{clamp}_{[0,1]}\left(t - 0.064 \sin\left[\pi S(0.68, 1, t)\right]\right),$$

$$F_{\mathrm{fold}}(s, t) = \mathrm{lerp}\left(F_{\mathrm{shoulder}}, E(s)^{1.78}, 0.58\, S(0.70, 1, t)\right),$$

$$F_{\mathrm{lip}}(s, t) = \mathrm{lerp}\left(E(s)^{1.18}, E(s)^{1.90}, 0.68\, S(0.70, 1, t)\right).$$

The sampler $\tau(t)$ keeps the lower curl inside the rounded hook before releasing to the flat tail. This repairs the pointy/squared side profile without authorizing a full-width tunnel. Candidate876 also keeps the crest-cap exception around the rounded lip:

$$C(t) = S(0.54, 0.66, t)\left[1 - S(0.82, 0.94, t)\right],$$

$$H_{\mathrm{cap}}(s) = \left(1 - S(0.20, 0.82, |s|)\right)^{0.78},$$

and blends the height span toward H_{cap} with weight $0.86C(t)$. This is not a tunnel relaxation. It is the blocker against the opposite failure Alan identified: a pointy fin or spike in the front/cross-section view. In the model path, the same localization principle appears through `transverseSupportMask`, `lipAwareRimMask`, `terminalLipSpanMask`, `shapeMask`, and the post-core taper terms.

Bowl/dimple blocker

The source blocker `blockSurfaceBowlPockets` runs before node emission. For each interior node with active neighboring height, it replaces a local low point by the minimum of its four direct neighbors:

$$z_{ij}^{(k+1)} = \max\left(z_{ij}^{(k)}, \min(z_{i-1,j}^{(k)}, z_{i+1,j}^{(k)}, z_{i,j-1}^{(k)}, z_{i,j+1}^{(k)})\right),$$

for up to six passes. This is a pre-render invariant sweep, not a visual cleanup. It catches bowl-like local minima, but it is not sufficient for the under-overhang support-neck failure; that failure also requires side/isometric throat sampling and human-facing render inspection.

Connected X-cell equations

For each node center c_n, the visible cell is one square body with four legs in directions $\{\mathrm{NE}, \mathrm{SW}, \mathrm{NW}, \mathrm{SE}\}$. For each opposite pair,

$$\|e_{\mathrm{NE}} - c_n\| = \|e_{\mathrm{SW}} - c_n\|, \qquad e_{\mathrm{NE}} + e_{\mathrm{SW}} = 2c_n,$$

and similarly for NW/SE. The same connector point g_{ab} is shared by exactly two adjacent cells:

$$e_{a,+} = g_{ab} = e_{b,-}, \qquad \#\{(n,d) : e_{n,d} = g_{ab}\} = 2.$$

The current source solves connector chains by alternating signs along each row/column family:

$$g_0 = f, \qquad g_k = 2c_k - g_{k-1} \quad (k > 0),$$

then chooses f by least-squares midpoint residual over the adjacent pair family. This preserves exact cell-level opposite-pair closure and exposes center-corridor drift as a diagnostic rather than hiding it. The checker then groups connector positions by physical coordinate, not only by id, so two connector ids occupying one point cannot create a silent four-cell joint.

Acceptance objective

WaveVis is not currently optimizing a single scalar objective. The useful mental model is a constrained inverse-design problem:

$$\min_{\theta,\mathcal{G}} \; w_r E_{\text{ref}}(\Phi_\theta, \Phi_{\text{ref}}) + w_s E_{\text{smooth}} + w_m E_{\text{mechanism}} + w_v E_{\text{view}}$$

subject to boundary pinning, topology completeness, connector occupancy, no bowl/dimple pockets, and public route verification. At this stage the objective is enforced as explicit gates rather than a continuous solver: reference match is judged on the rendered object; mechanism feasibility is checked in source; stale paper/public assets are treated as failures.

For a future continuous solver, the scalar terms should be reconstructed from the current gates rather than invented from scratch. A practical decomposition is:

$$\begin{aligned}
E_{\text{ref}} &= \lambda_c d_{\text{Fr}}(\Pi_{\text{side}}\Phi_\theta, \Gamma_{\text{side}}) + \lambda_o\, E_{\text{open}} + \lambda_t\, E_{\text{top}},\\
E_{\text{smooth}} &= \sum_{(i,j)} \left\|\Delta_x^2 p_{ij}\right\|^2 + \left\|\Delta_y^2 p_{ij}\right\|^2,\\
E_{\text{mechanism}} &= \sum_{n} \left(\epsilon_{\text{oppLen}}(n)^2 + \epsilon_{\text{oppCol}}(n)^2 + \epsilon_{\text{jointUse}}(n)^2 + \epsilon_{\text{anchor}}(n)^2\right),\\
E_{\text{view}} &= \epsilon_{\text{sideWall}}^2 + \epsilon_{\text{skinnySail}}^2 + \epsilon_{\text{frontPoint}}^2 + \epsilon_{\text{frontWall}}^2 + \epsilon_{\text{hiddenMech}}^2.
\end{aligned}$$

Here d_{Fr} is a curve-distance proxy between the side projection and the supplied side reference, E_{open} samples the underside throat below the lip for dimples, bites, bowls, and support-necks, and E_{top} rejects broad terminal blobs, triangles, and V fans in plan view. $E_{\text{frontPoint}}$ rejects a rounded-wave front view that collapses into a vertical fin; $E_{\text{frontWall}}$ rejects the opposite full-width wall. This follows the general inverse-design pattern of optimizing a physically meaningful state subject to explicit manufacturability and simulation constraints (*10*, *19–21*), but WaveVis should keep the gate form until the rendered artifact is stable enough that a scalar objective would not hide a visible failure.

Candidate iteration record

Every new geometry candidate must be traceable. The minimum record is:

$$\mathcal{R}_k = \{\text{candidate id, source diff, intended defect, metrics, render captures, accept/reject reason}\}.$$

The accept/reject reason must name the rendered shape, not only the code change. A valid note says, for example, "rejected: side open throat improved but top became a broad terminal blob." An invalid note says only "geometry check passed." Candidate848–Candidate875 are examples of why this record matters: some local gates passed, but browser renders still showed support ridges, broad top blobs, front-wall reads, pointy fin profiles, squared terminal feet, or support-cone reads. Candidate876 is the current local candidate: its side view follows the rounded supplied line and its checker/build pass, but the isometric cone/support-neck read is still open visual debt rather than accepted reference match.

3.4 Rigid controls

`position` and `steer` are post-shape placement controls:

$$\Delta x_{\text{position}} = 0.045\, L\, \text{clamp}(\texttt{position}, -1, 1),$$

$$\psi_{\text{steer}} = 45^\circ\, \text{clamp}(\texttt{steer}, -1, 1).$$

They must not modify the curated wave profile, width, curl, strain, or topology. Validation aligns geometries back to the canonical frame and checks that residual shape differences are near zero for position and bounded for steer.

3.5 Morph control

`morph` is not a shape recipe. It interpolates from rest to the completed target:

$$p_{ij}(m) = (1 - g(m))p^0_{ij} + g(m)p^*_{ij}.$$

The current growth function combines a sine ease with a faster visible growth curve so the wave grows naturally without a dead early range. Morph must preserve topology, connector closure, and the visible wave identity throughout the range. If `morph` = 0.5 produces zig-zags, overlaps, missing cells, or a side-wall-covered curl, that is a geometry failure.

3.6 Flat contribution and ground transition

`flatContribution` does not mean flatten the wave. It means share deformation with the surrounding sheet through a support apron. In accepted behavior:

- the main wave height and overhang reach stay within a small residual;
- the centerline profile remains essentially unchanged;
- surrounding flat areas participate more gradually;
- strain concentration is reduced or spread rather than hidden.

The old behavior `target` = lerp(`deformed`, `rest`, `flatContribution`) is explicitly rejected because it turns `flatContribution` into a flattening slider.

`smoothing`, shown historically as ground transition, broadens the support field and root ramp. It should recruit surrounding flat sheet into the slope while keeping the perimeter pinned and preserving the terminal lip.

3.7 Lip dip and terminal curl

`lipDip` maps to `overhangAngleDeg`. Neutral is 90°. High curl is 120°. The accepted behavior is not endpoint lowering. The wave should rise, crest, reach forward, curl downward, then return smoothly to the flat sheet without closing into a bowl. The current validation checks:

- the selected lip nose is a mid-height visible point below the last peak, not the near-floor return;
- the terminal slope is negative;
- the tip is forward of the crest and tucked under the shoulder by a bounded amount;
- the return advances toward the flat front instead of folding backward;
- the terminal cavity blocker, backfold blocker, and visible-dimple blocker remain true.

These checks are guards, not substitutes for the human-facing screenshot. A visible dimple, under-overhang pocket, side wall, or missing curl still means open work.

3.8 Rendering contract

`LatticeViewer3D.tsx` renders the readable surface and a connected X-cell mechanism from the actual lattice graph. In the current contract:

- `view=side` sets a side camera over the full three-dimensional linkage array;
- `view=front` sets a front camera over a crest-section window sampled from the same readable three-dimensional field, so the view inspects the rounded cross-section instead of projecting the whole side trace into a fin;
- `view=isometric` shows the full rectangular sheet in perspective while still making the curl and lip readable;
- the default product view uses a smooth gridded surface skin plus one-center four-leg X-cell leg lines, black square cell bodies, and spherical connector nodes;
- Side, Front, and 3D use view-specific mesh outlines instead of the previous SVG overlay, centerline comparison, or red terminal trace;
- each rendered X cell is one black square cell body with four visible center-to-connector legs;
- each connector is a spherical two-use physical node between adjacent cells;
- each connector has exactly two endpoint uses: one leg from each of those two centers;
- crossing diagonal families must stay visually and physically separate instead of collapsing into a four-cell joint;
- connector corridor drift is reported as a bounded diagnostic when exact cell-level opposite-pair closure moves a connector away from the old midpoint corridor;
- cells and X legs are display toggles, not repair paths, and their render scope remains covered by the geometry check.

If the renderer needs to hide rows to make the shape readable, the generator is still wrong.

4 Target outcome

4.1 Human-facing shape

The accepted target is a “flat sheet plus localized plunging wave”:

- the outer perimeter of the rectangular sheet remains flat and fixed;
- the wave emerges through a broad, smooth ramp rather than a vertical wall;
- the crest is rounded, with no top dent;
- the lip projects forward and curls downward;
- the open underside is visible from side view, not covered by a side wall;
- the shape fades back to flat in every direction;
- the array remains a full linkage field with nodes and legs, not a hidden or clipped shell.

Figure 7 gives the practical acceptance boundary. The shape must read as a local wave in a sheet, not as an extrusion. A single side profile can guide the curl, but the generated three-dimensional field must vary across span.

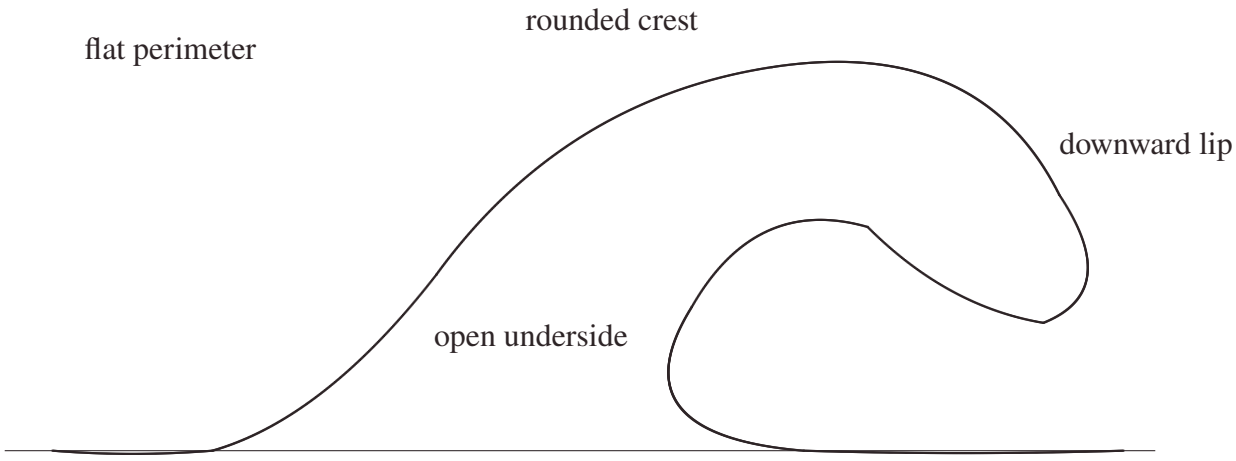

Figure 7. Desired side-profile shape as one constraint inside the full mechanism view. The drawing is not a literal mesh recipe and not an accepted output by itself. It defines a smooth back ramp, larger outer radius, smaller inner radius, curled lip, and curved interior throat; the three-dimensional solver must localize that profile in the middle of a full visible linkage sheet rather than sweep it as a constant tunnel or collapse the render to a trace.

4.2 Hard constraints

The WaveVis generator and renderer must preserve the following hard constraints:

1. **Flat perimeter.** Boundary nodes must remain on the rest sheet. No side, front, or rear perimeter should be pulled into the wave.
2. **Full array visibility.** The rendered artifact must show the actual linkage grid. Hiding or ghosting broken cells does not count as a fix.
3. **Connector closure.** A cell connector is an actual shared graph location. Lines should meet at nodes; apparent detached legs are a failure.
4. **No erratic legs.** Long arms, spikes, zig-zags, random backtracking, and visible overlaps indicate a broken geometry or render path.
5. **Smooth strain sharing.** Adjacent rows and columns should change gradually enough that the mesh reads as a continuous mechanism field.
6. **Localized wave.** The curl must be strongest near the center and fade laterally. A constant side-to-side tunnel is not the target.
7. **Full-view side semantics.** Side view shows the full three-dimensional render from the side. It must not be replaced by a profile-only debug view.
8. **No filler geometry.** Cosmetic walls, caps, planks, bridges, hidden couplers, or masks must not be used to conceal a bad mechanism.

5 Coordinate model

The simulator starts from a rectangular rest lattice. A node with row i and column j has rest position

$$p_{ij}^{0} = (x_j, y_i, 0).$$

The target is a displacement field

$$p_{ij}^{*} = p_{ij}^{0} + m\, D(x_j, y_i; \theta),$$

where $m \in [0, 1]$ is the morph amount and θ represents all shape parameters except rigid placement. The morph parameter only interpolates between rest and the finalized target. It must not change the target shape definition, remesh the lattice, or introduce a different topology.

The longitudinal coordinate $u \in [0, 1]$ measures progress through the wave region from rear/root to front/lip. The span coordinate $v \in [-1, 1]$ measures lateral distance from the centerline. A good WaveVis target uses both:

$$D(x, y) = A(u, v)\, P(u) + B(u, v)\, Q(u),$$

where $P(u)$ is the side-profile contribution, $Q(u)$ can include forward curl or secondary displacement terms, and A, B are smooth two-dimensional envelopes that go to zero at the sheet perimeter.

The key architectural rule is that $A(u, v)$ is not constant in v. A constant span envelope turns any attractive profile into a tunnel. A localized plunging wave requires $A(u, 0)$ large near the centerline and $A(u, \pm 1) \approx 0$ near the sides, with a monotone or smoothly unimodal falloff rather than a hard wall.

6 Generation pipeline

The target-building pipeline is:

1. sanitize rows, columns, visible sliders, hidden legacy shape parameters, display flags, and URL parameters;
2. build the full rectangular node grid;
3. compute the canonical rounded breaking-wave displacement from the supplied reference target;
4. apply the span envelope that localizes the wave in y;
5. preserve the boundary by forcing perimeter displacement to zero;
6. apply morph interpolation from rest to target;
7. apply rigid steer and position transforms after the shape is finalized;
8. build full rectangular edge and quad topology;
9. compute strain, bend, shear, dihedral, and display metrics;

10. render the full array, side view, 3D view, and top view without substituting fake surfaces.

The distinction between the supplied reference target and rigid placement parameters is essential. The visible app no longer exposes manual xz or xy profile editors. The source target owns the rounded breaking-wave profile. The user-facing sliders are intentionally narrow: scale uniformly resizes that target, morph blends from rest to the completed target, steer rotates the finished target around the anchor, and position translates the finished target horizontally. Neither steer nor position may feed back into profile curvature, curl, width, strain generation, or cell topology.

Stage	Contract	Failure caught
Target profile	Defines the desired side profile shape.	Blob, neck/head, hill without lip, top dent.
Span envelope	Localizes the wave in the sheet.	Swept barrel, side wall, U-shaped tube blocker.
Boundary clamp	Keeps the perimeter flat.	Pulled sheet edges, non-flat perimeter.
Morph	Interpolates rest to target only.	Half-morph mangling, topology change while sliding.
Topology	Keeps full node/edge/quad graph.	Missing grid, hidden cells, detached legs.
Renderer	Shows the actual mechanism.	Fake shell, hidden wall, side view as profile-only debug output.
Paper evidence	Includes only target references, accepted mechanism proofs, accepted current renders, or clearly labeled rejected diagnostics.	Stale candidate screenshots, misleading overlays, and internal trace diagrams appearing as authoritative figures.

Table 3. Pipeline stages and the failure modes each stage must prevent.

7 Supplied rounded target

The current visible app does not ask the user to draw a profile. That path created too much ambiguity and repeatedly made the simulator look like a swept tunnel or hand-edited cavity rather than a single coherent wave. The user-facing target is now driven by the supplied rounded breaking-wave lip/body profile: a localized ocean deformation with a broad rear ramp, rounded crest, forward-and-down lip, open underside, smooth lateral fade, and flat rectangular perimeter.

The curated target is a reference field, not a set of cell locations. The final cell grid is determined by row and column counts, then sampled against that target. A clean state must keep the full linkage array visible while making the centerline and lateral profile read as one plunging wave. The target must

never stitch the lip directly to the ground or draw a hard polygon boundary around a former editor curve, because that creates the wall, bowl, cavity, and silhouette-only failures this record explicitly rejects. The curl is also resolution-sensitive: the default reference-wave state keeps 112 columns, while reduced column counts are reserved for low-density mechanism inspection and must not be mistaken for a completed reference-shape view.

8 Slider semantics

Control	Intended effect	Must not do
`morph`	Blend rest sheet into the completed target.	Remesh, flip cells, create zig-zags, change profile identity.
`scale`	Uniformly enlarge or reduce the supplied rounded target.	Change steer, position, topology, or the target's wave identity.
`steer`	Rigid yaw rotation of the finished wave.	Bend, stretch, or reshape the wave.
`position`	Rigid horizontal translation of the finished wave.	Change profile, strain field, or curl.
display toggles	Surface, flat grid, cells, and X legs are display layers only.	Hide broken geometry or substitute a fake repair path.

Table 4. Visible control contract after removing manual profile editors and hidden shape sliders. The supplied rounded target is the default shape; visible controls must preserve that shape unless their narrow role explicitly changes scale, morph, steer, position, or display layers.

9 Breaking-wave geometry

The desired default profile is closer to a localized plunging wave than to a cone, cylinder, or arch. The useful approximation is a tapered, asymmetric surface whose highest crest sits behind the curl, whose outer radius is larger than the inner radius, and whose underside remains open. Along the centerline, a breaking wave profile can be represented as three joined curve families:

$$P(u) = \begin{cases} P_{\text{ramp}}(u), & 0 \le u < u_c, \\ P_{\text{crest}}(u), & u_c \le u < u_l, \\ P_{\text{lip}}(u), & u_l \le u \le 1. \end{cases}$$

The ramp rises from the sheet. The crest rounds over. The lip turns forward and downward. The derivative at the terminal lip should point downward when `lipDip` $> 90°$:

$$\left.\frac{dz}{dx}\right|_{\text{tip}} < 0.$$

The final point is allowed to be lower than the farthest horizontal reach. In a real breaking wave, the maximum reach can occur at the shoulder while the lip tip turns down from that shoulder. The downturn must remain open and forward-readable from the side; it must not fold into a closed pucker, dimple, bowl, or side-wall pocket. Treating the farthest-right point as the tip is the mistake that produces a straight falling chute, while overcorrecting into an under-tucked return is the mistake that produces the rejected cavity. The underside return must also advance back into the flat sheet after the lowest lip sample; a backward floor point is disallowed because it reads as a small lower-lip dimple even when it does not trip a larger bowl metric. Candidate785 was the previous recovery floor for this idea, not the accepted answer; the current acceptance test is still the rendered full mechanism with an open throat and no support-neck pocket.

The three-dimensional field must multiply this centerline motion by a span falloff:

$$E(v;u) = \exp\left[-\left(\frac{|v|}{w(u)}\right)^{\alpha}\right],$$

with $w(u)$ varying through the wave. The curl should have a smaller active width than the broad back ramp, so the lip is localized and the open underside remains visible from the side.

10 Mechanism topology

WaveVis uses a rectangular graph of nodes, horizontal profile edges, vertical span edges, and quads as the source lattice. The visible mechanism layer then derives a connected X cell at each lattice node. This is a kinematic-design problem, not a drawing style: linkage synthesis and symbolic kinematics treat joints, bars, and closure equations as the object being designed (*15*, *16*, *24*). The WaveVis renderer must therefore show the constraint truth rather than draw a smoother surrogate.

Each cell renders as a black square body with four visible legs that leave the same center. Opposite leg pairs are equal length and collinear through that center, so the center bisects both bars. Each leg terminates at a spherical connector node that is shared by exactly two adjacent cells. Crossing

diagonal families are separated into distinct physical joints so a crossing point cannot become a four-cell connector. The graph is not allowed to become isolated crossed marks, two nearby two-leg pseudo-cells, or a set of isolated rows.

The mechanism evidence from the earlier proof remains relevant as a warning against fake filler surfaces and impossible all-four-equal closure. The current mechanism rule is cell-level, not candidate-level: each cell has one square center and four straight legs, opposite pairs are collinear and equal length through that center, and each connector is a spherical physical joint used by exactly two adjacent cells. The old center-pair corridor is a drift diagnostic, not the hard pass condition (*25*).

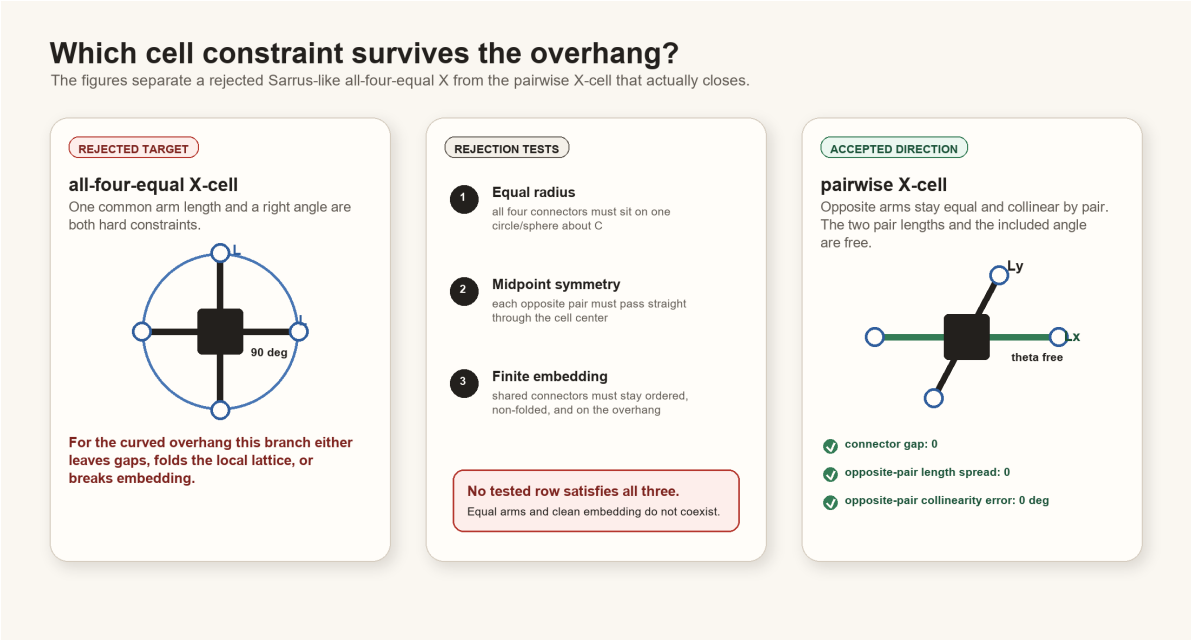

Figure 8. Mechanism contract inherited from the constraint proof and updated in the current renderer. WaveVis should preserve exact shared contact, one-center cells, and two-cell connector joints on adjacent-center spans rather than hiding infeasible geometry behind filler surfaces or a connector-ID count.

11 Rendering modes

WaveVis has three product views:

- **3D/isometric view** shows the full rectangular array in perspective.
- **Side view** is a camera view of the full three-dimensional render from the side and must show the curl.
- **Top view** shows the full square plan sheet with localized interior deformation and catches terminal caps, triangular fans, wedges, or cropped-patch regressions.

The view contract matters because a side view can hide the curl if lateral geometry forms a wall over the open underside. In that case the correct fix is to reshape the three-dimensional surface so the lateral envelope follows the wave contour and fades out, not to switch the side view into a profile-only debug output or hide the wall.

12 Verification gates

The generator should be checked with both numeric and visual gates. Numeric checks are not sufficient, but they catch repeated hard failures before visual review.

Gate	Measurement	Acceptance intent
Perimeter flatness	$\max_{\partial\Omega} \|p^* - p^0\|$.	Boundary displacement is zero or visually negligible.
Topology completeness	Expected node, edge, and quad counts.	Full rectangular sheet graph remains visible.
Connector closure	Edge endpoints meet their node coordinates.	No detached legs or fake bridges.
Lip curl	Crest-to-lip height gap, terminal slope, run gate, and cavity gate.	The lip curls down into a smooth tapered branch rather than ending as a hill, near-vertical chute, hooked tip, bowl, dimple, or cavity.
Span localization	Row-wise height profile across y.	Center strong, sides faded; no swept tunnel.
Morph continuity	Residual between adjacent morph samples.	No sudden jumps or topology flips.
Slider isolation	Aligned geometries across position/steer/width changes.	Rigid controls stay rigid; width stays span-only.
Visual side check	Browser side view screenshot or direct render inspection.	Curl visible in the full side view.
Defect repair loop	Confirmed visible defects re-enter source repair.	Acknowledging dimples, pockets, walls, or missing curl is not a stopping state.
Terminal side-profile gate	Literal readable profile and sampled model side trace.	Before visual QA, reject any lower-branch re-rise, under-lip valley rebound, lower-branch backtrack, abrupt terminal angle jump, or steep visible terminal descent.
Startup URL parser	Covers all public and legacy slider aliases.	A shared verification URL cannot silently load a different height, overhang, lip dip, width, or smoothing state.
Paper figure hygiene	Every figure is a target reference, accepted mechanism proof, current accepted render, or labeled rejected diagnostic.	Internal overlays, stale screenshots, and profile-only traces cannot become authoritative paper evidence.

Table 5. Verification gates. A build passing TypeScript is not enough; WaveVis must pass the rendered outcome Alan actually uses.

12.1 Current regression command set

The current source-level gate is:

```
npm run check:geometry
```

which expands to:

```
node scripts/check-geometry-invariants.cjs
node scripts/check-inverse-sheet.cjs
```

The inverse-sheet checker compiles the TypeScript geometry into a temporary CommonJS module and then evaluates the default state, low/high lip dip, morph, position, steer, flat contribution, smoothing, width, lateral taper, bowl-pocket, side-render, and startup-URL cases. The geometry-invariant checker covers mechanism-level consistency.

The current acceptance evidence expected from `scripts/check-inverse-sheet.cjs` includes:

- startup display defaults to `showSurface=true`, `showEdges=true`, `showNodes=true`, and `showHeatmap=false`;
- rendered black-square cell count equals the full rectangular grid count 4928;
- rendered center-to-connector X leg segment count equals the expected connector-use count 19400;
- every interior X cell has exactly four legs from one black square body;
- rendered spherical connector-node count equals the expected adjacent connector count 9700;
- each physical connector has exactly two endpoint uses;
- maximum X connector endpoint gap is zero;
- maximum opposite-pair length spread is zero;
- maximum opposite-pair collinearity error is zero degrees;
- maximum opposite-center residual is zero;
- connector corridor drift is bounded as a diagnostic rather than used as the pass condition;
- `sourceUsesCollinearEqualPairConnectors=true`;

- connected X-cell bodies stay close to the sampled wave surface rather than becoming a detached fake mechanism;
- no bowl-pocket count is reported for default, compact-user, and breaking-wide presets;
- the literal readable side trace, side/isometric readable profile, startup default, user morph half-lip case, and slider sweeps all pass the terminal and readable side-profile cavity blockers before browser screenshots are treated as useful evidence;
- `blockSurfaceBowlPockets` runs on the target grid before morphing and node emission, and the named 79-case slider sweep reports empty `badCases` arrays for the side-cavity and surface-bowl gates;
- side-overhang aspect stays in the downturned-lip band after the supplied-line update: startup default aspect between 1.35 and 3.05, compact user morph case between 1.45 and 3.15, and both with reach-to-height between 0.72 and 1.35;
- broad open-curl lip stats keep open-throat, no-dimple, no-under-overhang-bite, no-backfold, and return-to-flat checks true;
- those lip stats allow a wide rounded cap and tucked nose instead of forcing the old vertical-chute proxy;
- slider robustness exits cleanly for the named acceptance sweeps, while printed diagnostic subcases can still expose unsupported-resolution or terminal-angle failures that must not be described as accepted visual states.

These checks are designed around Alan's repeated failures. If a future change removes one of these cases because it is inconvenient, that is a regression unless the PDF and source explain an intentional replacement gate that still catches the same failure.

12.2 Rendered verification

Numeric checks must be followed by rendered verification. The minimum visual pass is:

1. render the exact user URL/preset, not a nearby default;
2. check `Side`: full 3D side camera, visible curl, no profile-only substitution, no wall covering the open underside, and no dimple or dark pocket under the overhang;

3. check `3D`: full-sheet perspective view, visible curl/overhang, localized center deformation, no swept tunnel, no hidden rows, and no underside bite hidden by the camera angle;
4. check `Top`: the sheet should remain square while the interior footprint localizes toward the curl; it must not become a constant extruded strip, broad terminal cap, clipped patch, or large triangular fan;
5. inspect cells/connectors at the curl for zig-zags, random long arms, detached legs, overlaps, or missing four-leg interior connectivity;
6. check lower `lipDip` values as well as 120°, because old low-lip cases became unstable even when max lip looked acceptable;
7. check `morph`=0, 0.5, and 1 because half-morph repeatedly exposed mangled intermediate states.

Screenshots are evidence only when they show the actual rendered canvas and the URL or visible controls identify the tested state. A screenshot of a nice profile line is not evidence that the full linkage array is correct.

13 Build, deploy, and publication runbook

13.1 Local development

Use the WaveVis app root:

`_apps/wavevis.aolabs.io/`.

Typical local commands are:

- `npm run dev` to run the Vite development server;
- `npm run check:geometry` to run geometry and mechanism checks;
- `npm run build` to build the static app;
- `npm run deploy` to publish the app's own GitHub Pages branch.

Do not manually edit `dist`. Build it from source.

13.2 Public fallback deployment

The route Alan can reliably use is `https://aolabs.io/wavevis/`. That route is served from the AO Labs hub/fallback repository, not directly from the WaveVis app repository. Therefore a public WaveVis update requires two publication steps when the fallback route is the user-facing route:

1. build and publish the WaveVis app;
2. copy the built `dist` contents into `_apps/aolabs-site/wavevis/`;
3. commit and push the AO Labs site mirror;
4. verify that `https://aolabs.io/wavevis/` serves the new bundle hash and the rendered artifact.

This is not optional. A source commit or WaveVis `gh-pages` deploy can still leave Alan looking at a stale `aolabs.io/wavevis/` bundle.

13.3 Custom-domain boundary

`https://wavevis.aolabs.io/` is the preferred branded route, but a hostname/certificate failure means it is not the verified artifact. The current Progress source separates `wavevis_home` from `wavevis_custom_domain`. Keep them separate. Do not claim the custom subdomain is fixed until a fresh request verifies the certificate and body from that hostname.

13.4 Progress logging

After meaningful WaveVis work, write a Progress event before final response when practical. The event must include:

- `source_ids`: usually `wavevis_home` and `wavevis_custom_domain`;
- Alan's complaint/request;
- the issue being solved;
- the Codex change;
- the observed public source change;

- provenance: thread, commits, checks, screenshots, and deployment basis;
- uncertainty: exact match not proven, custom domain stale, or any unverified surface.

Progress does not automatically ingest every active Codex chat. If a future WaveVis chat uses this PDF and changes the app, it must still write the event itself.

14 Current verified and open state

This section is intentionally blunt so a future chat does not have to infer whether the system is finished.

Verified current facts on July 1, 2026:

- The reliable public route is `https://aolabs.io/wavevis/`; the preferred HTTPS subdomain remains a separate certificate/live-route check.
- The Inverse Sheet tab is the first/default option; Double-Layer Sarrus is secondary.
- The public PDF route is `/wavevis/proofs/wavevis-system-architecture.pdf`.
- Side, Front, Top, and 3D are product view modes. Side is a full three-dimensional sheet view, not a profile-only debug trace.
- The current source tree preserves the one-center four-leg X mechanism, black square cell bodies, spherical two-use connector nodes, and the geometry checker that measures those invariants.
- Candidate876 passes the local inverse-sheet checker `scripts/check-inverse-sheet.cjs`; the passing gate includes the segmented rounded supplied side trace, terminal taper, lower-curl return, flat perimeter, no bowl/dimple pocket, no skinny front fin, no full front wall, and visible one-center four-leg X-cell mechanism.
- Candidate876 passes local `npm run build`; the built bundle is in `dist/`.
- Candidate876 browser captures exist in:

  ```
  _verification/
  browser-20260701-
  candidate876-broader-terminal/
  ```

  The folder contains side, front, top, isometric, and forced mechanism-on views.

- The wrong internal reference-overlay figure and irrelevant stills have been removed from tracked paper/public figure assets.

Open state that blocks completion:

- Candidate876 is source/build/PDF/live-route verified for publication, not final reference proof. The rebuilt PDF was copied into the public proof paths; the fallback and standalone HTTP routes served `index-SPBHkj_l.js` during the July 1, 2026 check.
- Candidate785 and Candidate841–Candidate875 are not final accepted states. They remain rejected or superseded diagnostics.
- The side profile has moved away from the rejected pointy profile, but the current isometric support-cone read still blocks an exact reference-match claim.
- Exact matching to the supplied breaking-wave line drawing, June 24 gridded reference, June 28 stage/cross-section reference, or retained ocean-curl photographs is not proven.
- This is a simulator and architecture record. It is not evidence that a physical lattice prototype has been fabricated or experimentally validated.
- If the live app, source, screenshots, Progress, or this PDF disagree, the current rendered artifact and source tree must be inspected and reconciled before any completion claim.

15 Failure modes

The most important failure classes are now explicit:

1. **Swept barrel.** One side profile is dragged sideways at constant span. The result reads as a tunnel, roof, half-pipe, or bridge.
2. **Side-wall cover.** The lip may exist in a center profile, but full side view shows a wall covering the open underside.
3. **Straight chute.** The crest drops nearly vertically to a flat shelf instead of curling forward and under.
4. **Blob or lump.** Excess smoothing or broad support creates a mound with no wave identity.
5. **Top dent.** The outer crest has a kink or concave dent where a smooth radius is required.

6. **Dimple, cavity, under-lip bite, or bowl.** The surface creates a pocket, pucker, enclosed hollow, under-overhang throat notch, or bowl-like depression rather than a plunging sheet.

7. **Dangling terminal connection.** The side view avoids the old dimple metric but leaves a visible lower terminal tongue or foot under the lip. This still fails because the reference curl has a smooth open throat and a flat sheet return, not a small support hook that visually wires the lip to the ground.

8. **Stale URL state.** A browser URL contains height, overhang, width, lipDip, or smoothing values, but startup ignores them and renders a different state.

9. **Zig-zag linkage.** Neighboring legs alternate direction or length erratically.

10. **Hidden mechanism.** Broken topology is ghosted, clipped, or replaced by a cosmetic surface.

11. **Wrong paper evidence.** A stale overlay, profile trace, internal debug diagram, or rejected candidate screenshot appears as if it is a correct reference or accepted result.

12. **Passive defect acknowledgement.** A visible problem is named in chat or recorded in this paper, but the generator and rendered artifact are not repaired and rechecked.

The solution direction is to simplify the target field while strengthening the gates: localized smooth wave first, full array second, mechanism-clean render third. Complexity should be added only if it improves one of those outcomes without breaking the others.

16 Materials and Methods

The implementation source is the WaveVis React and TypeScript app (*26*). The main geometry source is `src/latticeGeometry.ts`. The primary render source is `src/LatticeViewer3D.tsx`. Controls are declared in `src/TargetShapeControls.tsx`. Regression checks are run through `scripts/check-inverse-sheet.cjs`.

The model is deliberately written as an inspectable engineering simulator rather than a black-box optimizer. The validation style borrows from topology-optimization practice, where a compact code can still be valuable when its filters, constraints, and objective terms are explicit (*27*); from linkage and compliant-mechanism design, where target motion is inseparable from topology, joint placement, actuator choice, and manufacturable compliance (*12–14*, *18*); from inflatable and auxetic inverse

design, where a desired three-dimensional target must be produced by a planar pattern or cell field rather than by a decorative surface (*19–21*); and from mechanical metamaterials, where local cell rules can create global deformation modes but can also produce unintended instabilities if the cell rule is not made explicit (*22*, *23*, *28*). WaveVis currently encodes those lessons as hard source gates rather than as a mature continuous optimizer.

The public app serves a static build. The architecture PDF is placed in the public proofs directory as `wavevis-system-architecture.pdf` and linked from the WaveVis control headers. The old connector proof remains a source artifact for the mechanism decision, but the visible paper action now points to this architecture record because the current work is broader than connector assignment.

17 Discussion

The main design lesson is that WaveVis cannot be corrected by tuning only the side profile. The user-facing failure was three-dimensional: side walls, swept tubes, missing full-array linkages, broken side-view semantics, pointy front fins, and zig-zag legs. The architecture therefore now uses a single curated target contract rather than a manual editor contract. The sheet generator is responsible for creating a localized breaking-wave displacement field with lateral fade, boundary preservation, topology completeness, and a visible curled lip.

This distinction also clarifies future work. If the simulator produces a good center profile but a bad side view, the profile is only evidence that the curve exists somewhere. The real artifact still fails if the full three-dimensional render hides the curl or reads as an extrusion. Similarly, if the mesh is clean but the wave looks like a low mound, the mechanism is not enough. Both the shape and the linkage must be correct.

A newer lesson is that defect naming is not progress by itself. If a rendered state shows dimples, under-overhang pockets, side walls, or a missing curl, the correct workflow is source repair, geometry-check update, rendered screenshot verification, and public-bundle verification. The current checks name this directly through `noVisibleDimplePocket` and `readableSideProfileCavityBlock`: the lip must remain an open downturned branch with visible clearance and a return that advances into the flat sheet instead of folding into a pucker or dark throat bite. The same source-of-truth rule applies to URL state. Several correction screenshots used URLs that carried explicit height, overhang, width, lipDip, and smoothing values; ignoring those parameters caused the page to render a

different geometry than the requested test state. The startup parser is now part of validation through `startupUrlParamCoverage`. The architecture record is useful only if it changes the generator, state loader, and acceptance loop; it cannot substitute for a corrected human-facing WaveVis render.

18 Acknowledgements

This record was written to reduce repeated handoff burden during WaveVis development. The repeated complaints about side walls, missing cells, zig-zag legs, profile-view confusion, and swept-barrel geometry are treated as acceptance criteria rather than optional polish notes.

19 Funding

No external funding is claimed in this system architecture record.

20 Author contributions

Alan N. Pham defined the target artifact, accepted and rejected visual states, and the mechanism constraints. Codex generated this architecture record from the current WaveVis source and the active development contract.

21 Competing interests

The author declares no competing interests for this internal architecture record.

22 Data availability

The source data for this record is the WaveVis source tree, the Alan-supplied rounded side-profile reference, the retained ocean-curl references copied into the proof figure set, the local proof diagrams, and the current simulator behavior. The public PDF is served with the WaveVis static app.

23 Code availability

The relevant implementation files are `src/latticeGeometry.ts`, `src/LatticeViewer3D.tsx`, `src/TargetShapeControls.tsx`, and `scripts/check-inverse-sheet.cjs` in the WaveVis app repository.

24 Additional information

This document supersedes the old header link to the narrow connector-contact constraint proof as the primary public WaveVis PDF. The connector proof remains an implementation reference for mechanism feasibility, but the current public artifact needs the broader system architecture and outcome-verification contract.

25 References

1. D. Terzopoulos, J. Platt, A. Barr, K. Fleischer, Elastically Deformable Models. *ACM SIGGRAPH Computer Graphics* **21**, 205–214 (1987).
2. D. Baraff, A. Witkin, presented at the Proceedings of SIGGRAPH 1998, pp. 43–54.
3. E. Grinspun, A. N. Hirani, M. Desbrun, P. Schröder, presented at the Proceedings of the 2003 ACM SIGGRAPH/Eurographics Symposium on Computer Animation, pp. 62–67.
4. M. Bergou, M. Wardetzky, S. Robinson, B. Audoly, E. Grinspun, Discrete Elastic Rods. *ACM Transactions on Graphics* **27**, 63:1–63:12 (2008).
5. T. Liu, A. W. Bargteil, J. F. O'Brien, L. Kavan, Fast Simulation of Mass-Spring Systems. *ACM Transactions on Graphics* **32**, 214:1–214:7 (2013).
6. A. Witkin, M. Kass, presented at the Proceedings of SIGGRAPH 1988, pp. 159–168.
7. M. Müller, B. Heidelberger, M. Hennix, J. Ratcliff, Position Based Dynamics. *Journal of Visual Communication and Image Representation* **18**, 109–118 (2007).
8. O. Sorkine, M. Alexa, presented at the Proceedings of the Fifth Eurographics Symposium on Geometry Processing, pp. 109–116.
9. S. Bouaziz, S. Martin, T. Liu, L. Kavan, M. Pauly, Projective Dynamics: Fusing Constraint Projections for Fast Simulation. *ACM Transactions on Graphics* **33**, 154:1–154:11 (2014).
10. T. Du *et al.*, DiffPD: Differentiable Projective Dynamics. *ACM Transactions on Graphics* **41**, 13:1–13:21 (2022).
11. M. Skouras, B. Thomaszewski, S. Coros, B. Bickel, M. Gross, Computational Design of Actuated Deformable Characters. *ACM Transactions on Graphics* **32**, 82:1–82:10 (2013).

12. S. Coros *et al.*, Computational Design of Mechanical Characters. *ACM Transactions on Graphics* **32**, 83:1–83:12 (2013).

13. B. Thomaszewski *et al.*, Computational Design of Linkage-Based Characters. *ACM Transactions on Graphics* **33**, 64:1–64:9 (2014).

14. V. Megaro *et al.*, A Computational Design Tool for Compliant Mechanisms. *ACM Transactions on Graphics* **36**, 82:1–82:12 (2017).

15. L. Zhu *et al.*, LinkEdit: Interactive Linkage Editing Using Symbolic Kinematics. *ACM Transactions on Graphics* **34**, 99:1–99:8 (2015).

16. J. M. McCarthy, *Geometric Design of Linkages* (Springer, 2000).

17. M. P. Bendsøe, N. Kikuchi, Generating Optimal Topologies in Structural Design Using a Homogenization Method. *Computer Methods in Applied Mechanics and Engineering* **71**, 197–224 (1988).

18. O. Sigmund, On the Design of Compliant Mechanisms Using Topology Optimization. *Mechanics of Structures and Machines* **25**, 493–524 (1997).

19. M. Skouras *et al.*, Designing Inflatable Structures. *ACM Transactions on Graphics* **33**, 63:1–63:10 (2014).

20. M. Konaković-Luković *et al.*, Beyond Developable: Computational Design and Fabrication with Auxetic Materials. *ACM Transactions on Graphics* **35**, 89:1–89:11 (2016).

21. J. Panetta *et al.*, Computational Inverse Design of Surface-Based Inflatables. *ACM Transactions on Graphics* **40**, 40:1–40:14 (2021).

22. C. Coulais, E. Teomy, K. de Reus, Y. Shokef, M. van Hecke, Combinatorial Design of Textured Mechanical Metamaterials. *Nature* **535**, 529–532 (2016).

23. M. Kadic, G. W. Milton, M. van Hecke, M. Wegener, 3D Metamaterials. *Nature Reviews Physics* **1**, 198–210 (2019).

24. K. H. Hunt, *Kinematic Geometry of Mechanisms* (Oxford University Press, 1978).

25. A. N. Pham, *WaveVis overhang connector assignment: evidence rejecting all-four-equal X-cells*, Internal WaveVis constraint proof and simulator evidence figures, 2026.

26. A. N. Pham, *WaveVis inverse-sheet simulator source tree*, Local source files: src/latticeGeometry.ts, src/LatticeViewer3D.tsx, src/TargetShapeControls.tsx, scripts/check-inverse-sheet.cjs, 2026.

27. O. Sigmund, A 99 Line Topology Optimization Code Written in Matlab. *Structural and Multidisciplinary Optimization* **21**, 120–127 (2001).

28. K. Bertoldi, P. M. Reis, S. Willshaw, T. Mullin, Negative Poisson's Ratio Behavior Induced by an Elastic Instability. *Advanced Materials* **22**, 361–366 (2010).